

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.19

MIGRATIONS

 Copy as Markdown

Migrations

Serverpod comes with a migration system that helps you keep your database schema up to date as your project evolves. Database migrations provide a structured way of upgrading your database while maintaining existing data.

A migration is a set of database operations (for example, creating a table or adding a column) required to update the database schema to match the requirements of the project. Each migration handles both initializing a new database and rolling an existing one forward from a previous state.

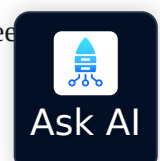
If you ever get out of sync with the migration system, repair migrations can be used to bring the database schema up to date with the migration system. Repair migrations identify the differences between the two and create a unique migration that brings the live database schema in sync with a migration database schema.

Opt out of migrations

It is possible to selectively opt out of the migration system per table basis, by setting the `managedMigration` key to false in your model. When this flag is set to false the generated migrations will not define any SQL code for this table. You will instead have to manually define and manage the life cycle of this table.

```
class: Example
table: example
managedMigration: false
fields:
  name: String
```

If you want to transition a manually managed table to then be managed by Serverpod you first need to set the `managedMigration` flag to `true`. Then you have two options:



- Delete the old table you had created by yourself, and this will let Serverpod manage the schema from a clean state. However, this means that you would lose any data that was stored in the table.
- Make sure that the table schema matches the expected schema you have configured. This can be done by either manually making sure the schema aligns, or by creating a [repair migration](#) to get back into the correct state.

Creating a migration

To create a migration navigate to your project's `server` package directory and run the `create-migration` command.

```
$ serverpod create-migration
```

The command reads the database schema from the last migration, then compares it to the database schema necessary to accommodate the projects, and any module dependencies, current database requirements. If differences are identified, a new migration is created in the `migrations` directory to roll the database forward.

If no previous migration exists it will create a migration assuming there is no initial state.

If the project has [client-side database](#) tables, matching SQLite migrations are also generated in the client package's `lib/migrations/` directory. They are applied on the device when the client database session opens.

See the [Pre-migration project upgrade path](#) section for more information on how to get started with migrations for any project created before migrations were introduced in Serverpod.

Force create migration

The command stops without creating a migration in two situations, and each has its own override flag:

- When no changes are detected between the latest migration and the schema the project requires, the command skips creating a migration. Pass `--empty` to create the migration anyway, for example as a starting point for [custom SQL](#).
- When the change carries a risk of destroying data, the command aborts with a warning. Pass `--force` to override the warning and create the migration.

```
$ serverpod create-migration --force
$ serverpod create-migration --empty
```

Tag migration

Tags can be useful to identify migrations that introduced specific changes to the project. Tags are appended to the migration name and can be added with the `--tag` option. A tag can contain only lowercase letters, numbers, and dashes.

```
$ serverpod create-migration --tag "v1-0-0"
```

This would create a migration named `<timestamp>-v1-0-0`:

```
|— migrations
|   └─ 20231205080937028-v1-0-0
```

Writing custom SQL

Migrations are plain SQL, so you can edit a generated `migration.sql` to do work the generator does not produce on its own, such as seeding reference data, backfilling a new column, or transforming existing data alongside a schema change.

Open the `migration.sql` file in the migration's directory and add your statements. Serverpod applies the file as-is when it rolls the database forward.

Custom SQL is worth reaching for when:

- Initial or reference data needs to be in place as soon as the schema exists.
- A schema change requires moving or transforming existing data, for example splitting one column into two.

You own the correctness of anything you add. Make sure it matches the schema at that point in the migration history and rolls forward cleanly from the previous migration. Serverpod does not validate hand-written SQL. Once a migration has been applied, treat it as immutable and create a new migration for further changes instead of editing it.

Migrations directory structure

The `migrations` directory contains a folder for each migration that is created, looking like this for a project with two migrations:

```
|— migrations
|   └─ 20231205080937028
|       └─ 20231205081959122
|           └─ migration_registry.txt
```

Every migration is denoted by a directory labeled with a timestamp indicating when the migration was generated. Within the directory, there is a `migration_registry.txt` file. This file is automatically created

whenever migrations are generated and serves the purpose of cataloging the migrations. Its primary function is to identify migration conflicts.

For each migration, five files are created:

- **definition.json** - Contains the complete definition of the database schema, including any database schema changes from Serverpod module dependencies. This file is parsed by the Serverpod CLI to determine the target database schema for the migration.
- **definition.sql** - Contains SQL statements to create the complete database schema. This file is applied when initializing a new database.
- **definition_project.json** - Contains the definition of the database schema for only your project. This file is parsed by the Serverpod CLI to determine what tables are different by Serverpod modules.
- **migration.json** - Contains the actions that are part of the migration. This file is parsed by the Serverpod CLI.
- **migration.sql** - Contains SQL statements to update the database schema from the last migration to the current version. This file is applied when rolling the database forward.

Apply migrations

During development

The `serverpod start` command handles migrations for you. Pending migrations are applied automatically on the first boot of a session. While the session runs, with its terminal focused:

- Press **M** to create a migration from your model changes and apply it.
- Press **P** to create a repair migration and apply it.
- Press **A** to retry when applying a migration fails.

Hold **Shift** with **M** or **P** to force the migration. See [Running your server](#) for the full set of shortcuts.

In production or CI

To apply migrations explicitly, start the server runtime with the `--apply-migrations` flag from your `server` package directory. Migrations are applied as part of the startup sequence and the framework asserts that each migration is only applied once to the database.

```
$ dart run bin/main.dart --apply-migrations
```

If a migration fails, or the database doesn't match the latest migration afterward, the outcome depends on the run mode:

- Any run mode except `development`: the server logs `Failed to apply database migrations.` or `Database does not match target state.` and keeps starting.
- `development`: the server exits with code 1.

Migrations can also be applied with the [maintenance role](#). The server applies the migrations and then stops, which suits CI jobs and other automated processes. Outside `development`, it can still exit with code 0 after a failed migration, so check the logs for failures.

```
$ dart run bin/main.dart --role maintenance --apply-migrations
```

If migrations are applied at the same time as repair migration, the repair migration is applied first.

On Serverpod Cloud

Serverpod Cloud applies pending migrations for you on every deploy. If a migration fails, the server logs `Failed to apply database migrations.` Fix the migration and redeploy. See [Cloud database](#) for the full flow.

Creating a repair migration

If the database has been manually modified the database schema may be out of sync with the migration system. In this case, a repair migration can be created to bring the database schema up to date with the migration system.

By default, the command connects to and pulls a live database schema from a running development server.

To create a repair migration, navigate to your project's `server` package directory and run the `create-repair-migration` command.

```
$ serverpod create-repair-migration
```

This creates a repair migration in the `repair-migration` directory targeting the project's latest migration.

A repair migration is represented by a single SQL file that contains the SQL statements necessary to bring the database schema up to date with the migration system.

WARNING

To restore the integrity of the database schema, repair migrations will attempt to remove any tables that are not part of the migration system. To preserve manually created or managed tables the [repair migration](#) needs to be modified accordingly before application.

Since each repair migration is created for a specific live database schema, Serverpod will overwrite the latest repair migration each time a new repair migration is created.

Migration database source

By default, the repair migration system connects to your `development` database using the information specified in your Serverpod config. To use a different database source, the `--mode` option is used.

```
$ serverpod create-repair-migration --mode production
```

The command connects and pulls the live database schema from a running server.

Targeting a specific migration

Repair migrations can also target a specific migration version by specifying the migration name with the `--version` option.

```
$ serverpod create-repair-migration --version 20230821135718-v1-0-0
```

This makes it possible to revert your database schema back to any older migration version.

Force create repair migration

The repair migration command aborts and displays an error under two conditions:

1. When no changes are identified between the database schema in the latest migration and the schema required by the project.
2. When there is a risk of data loss.

To override these safeguards and force the creation of a repair migration, use the `--force` flag.

```
$ serverpod create-repair-migration --force
```

Tag repair migration

Repair migrations can be tagged just like regular migrations. Tags are appended to the migration name and can be added with the `--tag` option.

```
$ serverpod create-repair-migration --tag "reset-migrations"
```

This would create a repair migration named `<timestamp>-reset-migrations` in the `repair-migration` directory:

```
|— repair-migration
|   └─ 20230821135718-reset-migrations.sql
```

Repair migrations directory structure

The `repair-migration` directory only exists if a repair migration has been created and contains a single SQL file containing statements to repair the database schema.

```
|— repair-migration
|   └─ 20230821135718-v1-0-0.sql
```

Applying a repair migration

The repair migration is applied using the server runtime. To apply a repair migration, start the server with the `--apply-repair-migration` flag. The repair migration is applied as part of the startup sequence and the framework asserts that each repair migration is only applied once to the database.

```
$ dart run bin/main.dart --apply-repair-migration
```

The repair migration can also be applied with the [maintenance role](#), where the server exits after applying it.

```
$ dart run bin/main.dart --role maintenance --apply-repair-migration
```

If a repair migration is applied at the same time as migrations, the repair migration is applied first.

Rolling back migrations

Utilizing repair migrations it is easy to roll back the project to any migration. This is useful if you want to revert the database schema to a previous state. To roll back to a previous migration, create a repair migration targeting the migration you want to roll back to, then apply the repair migration.

Note that data is not rolled back, only the database schema.

To roll back to a previous migration, first create a repair migration targeting the desired migration version:

```
$ serverpod create-repair-migration --version 20230821135718-v1-0-0 --tag "roll-back-to-v1-0-0"
```

Then apply the repair migration, any repair migration will only be applied once:

```
$ dart run bin/main.dart --apply-repair-migration
```

Troubleshooting

Check the flag name when migrations do not apply. The flag is `--apply-migrations`, plural. Passing the singular `--apply-migration` fails to parse, so Serverpod logs **Failed to parse command line arguments. Using default values.** and falls back to defaults. Migrations are not applied, and any other flags on the same command are dropped as well. If migrations are not running, check the flag spelling first.

Deleting a migration does not undo it. Removing a migration directory does not roll the schema back. To undo a migration, create a repair migration targeting the version you want to return to and apply it, as described in [Rolling back migrations](#). Only the schema is reverted; data is not.